

Legal Document QA Assistant

INF375 Artificial Intelligence Final Project Report

Live demo: <https://madot12-legal-doc-qa-assistant.hf.space>

This project implements an evidence-grounded question answering system for legal texts and contracts. The system accepts legal clauses or uploaded documents, retrieves the most relevant context, and returns an answer together with supporting evidence and confidence scores.

Dataset questions

75

Evidence accuracy

100%

Average token F1

60%

Project Team

230103109 - Madi Zhazykenov

230103225 - Danial Maksatuly

230103382 - Sherzodbek Saydakhmedov

Problem Statement

Legal contracts are long, dense, and difficult to scan manually. A user may need to answer targeted questions such as whether assignment is allowed, how termination works, or what confidentiality obligations apply. The project goal is to reduce manual search effort by building a reproducible NLP pipeline that finds relevant clauses and extracts concise answers.

AI Technique Used

- Retrieval-Augmented Question Answering: retrieve relevant clauses first, then extract an answer from that evidence.
- Baseline: BM25 keyword retrieval plus transparent lexical answer extraction.
- Transformer extension: dense sentence-transformer retrieval and a fine-tuned Legal-BERT extractive QA reader.

Dataset Description and Methodology

Dataset Description

The project uses a self-created legal QA CSV dataset with manually prepared contract-style clauses, questions, expected answers, and evidence spans. The current dataset contains 75 question-answer examples across multiple legal clause categories. The application also supports uploading custom QA CSV files and loading CUAD-style legal QA data.

- Self-created examples cover confidentiality, payment, termination, data protection, lease, software service, and consulting clauses.
- The dataset includes hard negative no-answer questions whose wording overlaps with real contract clauses.

System Pipeline

- Document loading supports TXT, Markdown, PDF, and DOCX files.
- Chunking splits legal text into traceable passages with source metadata and section context.
- Retrieval ranks candidate clauses using BM25 or optional dense sentence-transformer embeddings.
- Answering uses either a lexical baseline or a Hugging Face extractive QA reader.
- The Streamlit interface exposes model settings, top-k retrieval, dataset upload, and baseline comparison.

Implementation Details

The code is organized into separate modules for loading, tokenization, chunking, retrieval, reading, and orchestration. This structure keeps the project reproducible and makes it possible to compare baseline retrieval against stronger transformer-based settings.

- Improved chunking keeps numbered legal headings attached to the following clause text.
- Transformer answer spans are post-processed to trim extra punctuation and whitespace.

Evaluation and Results

Evaluation Setup

Evaluation was run over the local QA CSV. For each question, the system retrieved the top three chunks and generated an answer. A prediction was counted as correct when the expected answer or expected evidence span appeared in the returned answer/evidence text. Token F1 was also computed between the expected answer and predicted answer.

Correct answers

75/75

Answer match

40%

Evidence match

77%

CSV Results Snapshot

Method	Correct	Accuracy	Evidence	Token F1
bm25+lexical	75/75	100.0%	77.3%	60.5%
dense+lexical	73/75	97.3%	74.7%	59.5%
bm25+transformers	75/75	100.0%	77.3%	80.8%
dense+transformers	74/75	98.7%	76.0%	77.9%
bm25+legal-bert	75/75	100.0%	77.3%	63.2%

Error Analysis

- Answerability detection improved the No-Answer category to 17/17 correct, 0 errors, accuracy 100%.
- All answerable clause categories were retrieved correctly in the current CSV evaluation.
- This shows that the system now finds stated clauses and rejects unsupported questions on the project dataset.

Challenges, Limitations, and Improvements

Results Interpretation

The baseline performs strongly on the prepared dataset because the questions are tightly grounded in specific legal clauses and the expected evidence spans are present in the indexed text. This validates the retrieval and evidence-grounding workflow, while still leaving room for deeper transformer evaluation on larger public legal QA datasets.

Challenges and Limitations

- The self-created dataset is useful for demonstration, but still small for robust model claims.
- The lexical baseline can copy relevant evidence, but it does not deeply reason over multiple clauses.
- Transformer and dense embedding modes require optional model downloads and stronger runtime resources.
- The current evaluation uses evidence matching and token F1.

How the Project Was Strengthened

- Expanded the manually labeled dataset to include more contract types and harder no-answer questions.
- Improved retrieval input quality with section-aware legal chunking.
- Added answer post-processing for transformer spans to remove noisy boundary punctuation.
- Added answerability detection so unsupported questions return Not found instead of forced answers.
- Fine-tuned nlpaueb/legal-bert-base-uncased and saved the project checkpoint under models/legal-bert-qa.
- Added reproducible dataset-level evaluation with CSV outputs and JSON summary metrics.
- Added same-split method comparison across BM25, dense retrieval, lexical QA, and transformer QA.
- Added clause-type error analysis to show where the system fails, especially no-answer questions.